Project 5 — ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker

Title: ContentForge Pro Production Ready Content Pipeline With API Database And Docker

Type: Multi-agent service (Project 4's 5 agents, behind an API) · **Stack:** Python, FastAPI, SQLite, Docker, pytest, LangGraph · **Level:** Upper-Intermediate

Tagline: ContentForge (Project 4), served as a real API — every run saved, packaged in a container, rate-limited, and guarded by tests that catch a bad change before it ships.

> This is the **reading guide** — the story, the folder structure and why each part exists, setup, the 4 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker/`.

Charter (what this project is)

Project 4, made production-ready: served over FastAPI, every run saved to SQLite, packaged with Docker, rate-limited, and covered by tests at every layer — API tests with the team faked, and an eval suite that scores the real articles. **No new agents are built here** — Project 4's code is copied in unchanged. This project is about running those agents safely for other people.

Jump to: Read first · Project structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4 · Run, test, configure · Checklist · Publish

The Story — what you're actually building

Picture ContentForge as a very capable employee who only works if you sit at their desk and run their code by hand. They do good work, but nobody else can use them: there's no phone number to call, no record of what they did, and no way to trust them with many callers at once.

Project 5 turns that employee into a service. Step 1 gives it a phone number — an HTTP address anyone can call with a topic. Step 2 gives it a filing cabinet — every run saved to a database, findable by ID. Step 3 puts it in a shipping container (Docker), so it runs the same on any machine,

and adds a bouncer at the door (a rate limit). Step 4 adds a quality inspector — a test suite that notices when a change makes the articles worse.

None of these steps touch the five agents. The difference is the safety net around them: "a cool thing on my laptop" versus "a service I could hand to someone else".

What you're building, in one line: Project 4's 5-agent graph behind a FastAPI service, backed by SQLite, packaged with Docker, and guarded by API tests and an eval suite.

Why this needs to exist: a multi-agent system that only runs from a terminal can't be called by a web page, another service or a teammate — and without saved runs and tests, nobody can tell what went wrong, or whether a change made it worse.

When you'd reach for this at a real job: whenever a working prototype must become a service other people or systems call — this is what "shipping" an agent looks like.

How it works, mechanically: a `POST /run-task` route calls **one** function, `run_pipeline(topic)`, which runs Project 4's graph. The route saves a `"started"` row first and updates it after, tags every log line with a request ID, and turns a crash into a plain 500. Docker packages it with the key passed in only at run time. pytest checks the API with the team faked; an eval suite scores the real articles.

Why not keep running it as a script? Then only you can use it, nothing is recorded, and "it works on my machine" is the only promise. **Why not check changes by eye?** A change that makes articles a bit worse passes a quick look — an automated suite with a pass bar doesn't.

Where This Fits in the 5-Project Arc

```
Project 1                → 2 agents, Python router, raw SDK
Project 2                → 2 agents, hand-built agent loop, tools
Project 3                → 3 agents, LangGraph, RAG + human approval
Project 4                → 5 agents, one Supervisor directs the rest
Project 5 (you are here) → Project 4, made production-ready
                           (API + database + Docker + tests)
```

New here: nothing about the agents — everything **around** them. Project 4's code is the starting point, copied in unchanged.

What to Read First (docs this project uses)

You need to understand…	Read this	Used in
A route, `response_model`, `TestClient`, `APIRouter`	Doc12 — health route	Step 1
400 vs. 500, `logger.exception`, request IDs	Doc12 — chat route	Steps 1, 2
Saving runs, `GET /runs/{id}` with a 404	Doc12 — SQLite persistence	Step 2
`INSERT` → `lastrowid` → `UPDATE`, `with conn:`	Doc12 — validation 422	Step 2
`Dockerfile`, `.dockerignore`, the startup check	Doc12 — dead connection and container	Step 3
The whole API and the rate limiter, as a practice run	Doc12 Build Task	Steps 1-3
Test layers, faking the model	Doc13 — testing layers	Steps 1, 4
`monkeypatch`	Doc13 — test a tool without the LLM	Step 1
pytest `assert` tests	Doc13 — a normal, exact test	Step 4
Property checks, not exact wording	Doc13 — fix a flaky test	Step 4
A judge with written rules	Doc13 — score an answer	Step 4
Proving the suite catches a regression	Doc13 — regression catch	Step 4
The whole suite, as a practice run	Doc13 Build Task	Step 4
The team you're wrapping	Project 4	Setup

Reused code: all of Project 4, unchanged. `db/database.py`, the rate limiter, the request-ID middleware, the `Dockerfile` and `.dockerignore` are Doc12's Build Task (Intermediate); `judge.py`, `run_eval.py` and `regression_check.py` are Doc13's Build Task (Intermediate), pointed at `run_pipeline()`.

Project Structure (the professional layout)

Same rule as before: only folders with a real job. New here: `api/` (the web layer), `db/` (the run history), `evals/` (article quality), `pipeline.py` (the one door into the team), `tests/api/`, and the two Docker files at the root.

Names: the repo folder vs. the Python package

- **Repo folder:** `ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker/` — already created, and the exact GitHub repo name.
- **Python package:** `src/`, right next to `main.py` — the same as Project 4. Project 4's files come in unchanged: every `from src...` import keeps working, and the new code sits beside it (`from src.api.routes import router`).
- **Files:** `snake_case.py`; test files start with `test_`; the app uvicorn starts is `api/main.py` — the terminal app stays `main.py` at the repo root.

The final layout (after Step 4)

```
<repo folder>/   (ContentForge-Pro-...-API-Database-And-Docker/)
├── data/
│   └── sample/
│       └── facts.json          the facts Research can look up
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── analysis_agent.py   run_analysis(), analysis_node
│   │   ├── research_agent.py   run_research(), research_node
│   │   ├── reviewer_agent.py   run_reviewer(), reviewer_node
│   │   ├── supervisor.py       decide_next(), supervisor()
│   │   └── writer_agent.py     run_writer(), writer_node
│   ├── api/
│   │   ├── __init__.py
│   │   ├── main.py             the app uvicorn starts       (Doc12)
│   │   ├── routes.py           /health, /run-task, /runs/{id}
│   │   └── schemas.py          TaskRequest, TaskResponse    (Doc12)
│   ├── db/
│   │   ├── __init__.py
│   │   ├── database.py         save and fetch runs (SQLite) (Doc12)
│   │   └── models.py           Run: one saved run's shape   (Doc12)
│   ├── evals/
│   │   ├── __init__.py
│   │   ├── judge.py            score_with_judge()           (Doc13)
│   │   ├── regression_check.py breaks a prompt, checks drop
│   │   ├── run_eval.py         runs the suite, saves report
│   │   └── tasks.py            10 fixed topics and checks   (Doc13)
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              get_model()              (Project 3)
│   ├── prompts/
│   │   ├── analysis.txt
│   │   ├── research.txt
│   │   ├── reviewer.txt
│   │   └── writer.txt
```

```
│   ├── tools/
│   │   ├── __init__.py
│   │   └── lookup.py           lookup_facts (@tool)
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py           get_logger()                (Doc01)
│   │   └── prompts.py          load_prompt(name)        (Project 1)
│   ├── __init__.py
│   ├── config.py               Config, load_config()       (Doc01)
│   ├── exceptions.py           MissingConfigError          (Doc01)
│   ├── graph.py                build_graph(): 5 nodes, 1 edge
│   ├── pipeline.py             run_pipeline(): the one door in
│   ├── schemas.py              ReviewVerdict (score, feedback)
│   └── state.py                TeamState, start_state()
├── tests/
│   ├── api/
│   │   └── test_routes.py      the API, with the team faked
│   ├── integration/
│   │   ├── test_judge.py       real API calls
│   │   └── test_live_agents.py real API calls
│   └── unit/
│       ├── test_config.py      no API key needed
│       ├── test_lookup.py      no API key needed
│       └── test_supervisor.py  no API key needed
├── .dockerignore               keeps .env and runs.db out of image
├── .env                        your real key — never committed
├── .env.example                the setting names, no real values
├── .gitignore                  keeps .env, .venv/, runs.db out of git
├── Dockerfile                  builds the API's container image
├── LICENSE                     lets others reuse your code (MIT)
├── main.py                     the terminal app (from Project 4)
├── README.md                   the repo's front page (copied in at end)
└── requirements.txt            the libraries, for pip install -r
```

Files the app makes as it runs — `runs.db` (Step 2) and `report.json` (Step 4) — are in `.gitignore` and `.dockerignore`.

Why each folder exists

- **`data/sample/`** — Project 4's fact store, unchanged.
- **`src/`** — Project 4's package, unchanged, plus `pipeline.py`: the one function the API and the evals call to run the team. **Why:** a test swaps one function for a fake, and nothing outside the package touches the graph's state.
- **`agents/`**, **`tools/`**, **`prompts/`**, **`models/`**, **`utils/`** — Project 4's, unchanged.

- **`api/`** — the web layer: the app, the routes, and the request/reply shapes. **Why:** the HTTP rules (status codes, rate limit, request IDs) live apart from the agents.
- **`db/`** — every SQL statement behind small functions, plus a saved run's shape. **Why:** the routes read like a story, and the SQL is in one place.
- **`evals/`** — the fixed topics, the judge, the runner and the regression check. **Why:** article quality is measured with the real model, on purpose — not on every save.
- **`tests/api/`** — every route and status code, with the team faked by `monkeypatch`. No model calls, no cost.
- **`tests/integration/`** — the real team and the real judge. Costs money, so it's run on purpose.
- **`tests/unit/`** — the config, the lookup tool and the routing rules. No key, no cost.

Why each root file exists

- `.dockerignore` — keeps `.env` and local data out of the image.
- `.env` — your real API key; never committed.
- `.env.example` — which settings to create, with no real values.
- `.gitignore` — keeps secrets, the venv, `runs.db` and `report.json` out of git.
- `Dockerfile` — the recipe that builds the API's image.
- `LICENSE` — the MIT license text, created in Setup. Without one, nobody may legally reuse your code.
- `main.py` — Project 4's terminal app, still runnable with `python main.py`.
- `README.md` — the repo's front page on GitHub.
- `requirements.txt` — the libraries, plus the test tools, for `pip install -r`.

Why each file exists (what and why)

Every file you'll write, in one place — the same What/Why list the documents give after each tree. Project 4's files are listed in its own README; here are the ones Project 5 adds, plus the shared entry points.

- `data/sample/facts.json`
  - **What:** A JSON object mapping each topic to a list of short facts, each ending with a `[source]` tag. (Project 4's, unchanged.)
  - **Why:** You know these facts, so an invented fact in the article is easy to spot.
- `api/main.py`
  - **What:** Creates `app = FastAPI(...)`, calls `load_config()` at startup, includes the router, and adds `add_request_id`, a middleware that makes a short request ID, puts it on `request.state`, and returns it in the `X-Request-ID` header.
  - **Why:** uvicorn needs one app object to run. A missing key stops the server at startup, and every log line of a request can be found by its ID.
- `api/routes.py`

  - **What:** An `APIRouter` with three routes: `GET /health`; `POST /run-task`, which checks the rate limit, rejects a blank topic with 400, saves a "started" run, calls `run_pipeline()`, and on success saves the result — on a crash it marks the run failed, logs with `logger.exception`, and returns a plain 500; and `GET /runs/{run_id}`, which returns the run or a 404. Plus `check_rate_limit(caller_key)`, which raises 429.
  - **Why:** These are the addresses other programs call. Each failure gets the right HTTP status instead of a crash or a leaked traceback.
- `api/schemas.py`
  - **What:** Two Pydantic classes: `TaskRequest` (`topic: str` with a min and max length) and `TaskResponse` (`run_id`, `status`, `accepted`, `reason`, `article`).
  - **Why:** FastAPI checks every request against these shapes, so bad input gets a clear 422 instead of reaching the agents.
- `db/database.py`
  - **What:** `DB_PATH`, the `CREATE TABLE` SQL, and five functions: `get_connection()`, `insert_started_run(conn, topic)` (returns the new id from `lastrowid`), `finish_run()`, `fail_run()`, and `fetch_run(conn, run_id)` (returns a dict or `None`).
  - **Why:** Every run is saved with its status, so you can look it up later — even after a restart — and see which runs failed.
- `db/models.py`
  - **What:** One Pydantic class, `Run`: the fields of one saved run (`id`, `input`, `output`, `log`, `status`, `created_at`).
  - **Why:** The database code and the `GET /runs/{id}` response agree on the same fields.
- `evals/judge.py`
  - **What:** A `JudgeResult` dataclass (`verdict`, `reason`), `call_model(prompt)`, `build_judge_prompt(task_input, output, rules)`, and `score_with_judge(...)`, which reads PASS or FAIL from the first line of the reply.
  - **Why:** Some qualities, like clarity, can't be checked with exact code. A judge model with written rules can score them.
- `evals/regression_check.py`
  - **What:** `PASS_BAR`, a `SABOTAGE` text, `score(passed_by_id)`, `find_newly_failed(baseline, after)`, and `run_regression_check()`: run the suite, break one prompt on purpose, run it again, and report the drop.
  - **Why:** It proves the eval suite really catches a bad change. An eval that can never fail proves nothing.
- `evals/run_eval.py`
  - **What:** `run_one_task(task)` runs the pipeline and checks the article one of three ways (no article expected, an expected text, or the judge), and `run_eval_suite()` runs all tasks, prints the pass count and saves `report.json`.
  - **Why:** It gives one score for the whole system, so you know whether a change made it better or worse.

- `evals/tasks.py`
  - **What:** Rule constants and a `TASKS` list of 10 dicts: each has an `id`, a `topic`, and either an `expected` text, `expect_no_article`, or judge `rules`.
  - **Why:** The same fixed test set every time, so two versions of the system can be compared fairly.
- `pipeline.py`
  - **What:** `steps_from_log(routing_log)` returns just the step names, and `run_pipeline(topic)` builds the graph, runs it with `start_state(topic)`, and returns a small dict: `article`, `accepted`, `reason`, `steps`.
  - **Why:** The API and the evals both call this one function. Without it, each would build and read the graph in its own way.
- `__init__.py` (every code folder)
  - **What:** An empty file. You create one in every folder that holds Python code — nothing goes inside it. (The new `api/`, `db/` and `evals/` need one too.)
  - **Why:** Python only treats a folder as a package when this file is there. Without it, an import like `from src.<folder> import ...` fails with `ModuleNotFoundError`.
- `main.py`
  - **What:** Project 4's terminal app: `main()` loads the config, builds the graph, reads a topic and prints the report. Copied in during Setup, unchanged.
  - **Why:** The terminal app still works; the API is a second way in, not a replacement.
- `tests/api/test_routes.py`
  - **What:** pytest test functions that use `TestClient(app)` and `monkeypatch` to swap `routes.run_pipeline` for a fake, then `assert` the status code and body for each case (each step adds asserts or tests; names never change): health, a normal run that gets saved, a bad body (422), a blank topic (400), a crash (500), a missing run (404), the request ID header, and the rate limit (429).
  - **Why:** The API's behavior is tested quickly and for free, without any model calls.
- `tests/integration/test_judge.py`
  - **What:** A pytest test that gives the judge a known-good and two known-bad articles and `assert`s it passes the first and fails the others.
  - **Why:** You can only trust the judge's scores after checking it on examples where you already know the answer.
- `tests/integration/test_live_agents.py`
  - **What:** Project 4's plain OK/WRONG script against the real API — the Writer, Reviewer, Research, Analysis and the whole team. (Copied in during Setup, unchanged.)
  - **Why:** It proves the agents still work end to end in their new home.
- `tests/unit/test_config.py`, `test_lookup.py`, `test_supervisor.py`
  - **What:** Project 4's plain OK/WRONG unit scripts — the settings, the lookup tool and every routing rule. (Copied in during Setup, unchanged.)

- **Why:** Fast, free checks you run after every change, with no API key.

How imports work in this layout

- Every folder with Python files gets an empty `__init__.py` (the new `api/`, `db/` and `evals/` too). It marks the folder as a package, so `src.db.database` is importable.
- Python looks for imports in the folder you **run from**. `main.py` and `src/` sit side by side in the repo folder, so imports like `from src.api.routes import router` are found there.
- uvicorn gets the app by its import path from the repo root: `uvicorn src.api.main:app`.
- Tests live in sub-folders: run the plain scripts as modules (`python -m tests.unit.test_config`) and point pytest at a folder (`pytest tests/api`). Both are run from the repo root, so `src` is found.
- Always run commands from the **repo root** — the prompts, `data/sample/facts.json` and `runs.db` are all found from there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
Postgres, `docker-compose.yml`	One container and one SQLite file are enough	When several copies must share data (Doc12)
Redis for the rate limit	One process; the limit's weakness is written in the code	When you run several copies
A job queue for long runs	A run takes seconds, not minutes	Doc12's "Later" topics
Login / API keys for callers	A portfolio API, not a public product	Doc16 (security trade-offs)
CI/CD, a release gate, rollback	The suite runs by hand here	Doc19
`conftest.py`, fixtures	One small helper, `use_fake_team()`, does the job	When the test suite grows
Caching	Every request should get a fresh article	Doc12's caching topic
A frontend	`/docs` lets anyone try the API in the browser	Streamlit, when a project needs one

How the structure grows, step by step

Step	New files	Changed files
Setup	root files; Project 4's `src/`, `data/`, `tests/` and `main.py` copied in	—

Step	New files	Changed files
1	`pipeline.py`, `api/` (`__init__.py`, `main.py`, `routes.py`, `schemas.py`), `tests/api/test_routes.py`	—
2	`db/` (`__init__.py`, `database.py`, `models.py`)	`api/schemas.py`, `api/routes.py`, `api/main.py`, `test_routes.py`
3	`Dockerfile`, `.dockerignore`	`api/routes.py`, `test_routes.py`
4	`evals/` (`__init__.py`, `tasks.py`, `judge.py`, `run_eval.py`, `regression_check.py`), `tests/integration/test_judge.py`	—

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files. The same steps as Projects 1-4 — plus copying Project 4's code in.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`):

```
cd project_5_contentforge_pro_production
# the \ joins the two lines into one folder name
cd ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-\
Database-And-Docker
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Copy your finished Project 4 in — its `src/`, `data/`, `tests/` folders and `main.py`, unchanged:

```
# the \ joins the three lines into one path
P4=../../project_4_contentforge_multi_agent/\
ContentForge-Research-Write-Review-Team-Turns-Any-Topic-\
Into-A-Publish-Ready-Article
cp -r "$P4/src" "$P4/data" "$P4/tests" "$P4/main.py" .
```

3. Create `.gitignore` **before** your first commit:

```
# secrets — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
# Python's compiled cache files
__pycache__/
*.pyc
# pytest's own cache folder
.pytest_cache/
# the run history (Step 2) — local data, not code
runs.db
# the eval report (Step 4) — made fresh on every run
report.json
```

4. Create `.env` and `.env.example`, the same as before:

```
# .env — never commit this file
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

```
# .env.example — commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

5. Create `requirements.txt`:

```
fastapi>=0.110
langchain-core>=1.0
langchain-openai>=1.0
langgraph>=1.0
pydantic>=2.0
python-dotenv>=1.0
uvicorn[standard]>=0.29
# for the tests: TestClient runs on httpx
httpx>=0.27
pytest>=8.0
```

New since Project 4: `fastapi` (the API), `uvicorn` (the web server that runs it), `httpx` (what `TestClient` runs on) and `pytest`. SQLite needs nothing — `sqlite3` comes with Python. You'll also need Docker installed for Step 3 — a program on your computer, not a pip package.

6. Install, and check that Project 4 still runs in its new home:

```
pip install -r requirements.txt
python main.py           # Project 4's terminal app
```

Check it worked: from the repo folder, `python -c "import src; print('ok')"` prints `ok`, and `python main.py` writes an article, exactly as in Project 4.

7. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Why This Project Is Good for Your Portfolio

- **What problem it solves:** turns Project 4 from "works when I run it" into a service someone else can call, trust with real traffic, and debug afterwards.
- **Why it matters:** this is the project you link, not just describe — an API with a clear contract, a saved history, a clean Dockerfile, and tests that would catch a real regression.
- **When you'd build something like this at a job:** this *is* the everyday shape of production AI work — wrap an agent system in a service, save its history, test it, and defend each trade-off.
- **How it's built:** FastAPI routes on an `APIRouter`, Pydantic shapes, SQLite with a 3-state run status, request-ID logs, a per-caller rolling-window rate limit, Docker with secrets only at run time, pytest with a faked team, and an LLM-judge eval suite with a regression check.

Problems you'll likely run into, and how to fix them:

Problem	Fix
`MissingConfigError` when the tests start	The app refuses to start without `OPENAI_API_KEY` — keep it in `.env` (the API tests fake the team, so any value works)
An API test calls the real model	Swap `routes.run_pipeline`, the name the route uses — not `pipeline.run_pipeline`
Tests get a surprise `429`	Reset `routes.request_times` in every test (`use_fake_team()` does it)
A client sees a traceback or `str(exc)`	Catch at the route, `logger.exception()` on the server, a plain 500 to the client
A crashed run leaves no record	Save the `"started"` row in its own `with conn:` **before** the team runs; mark it `"failed"` on a crash
`curl` gets "connection refused" from the container	`--host 0.0.0.0` in `CMD`, and `-p 8000:8000` on `docker run`
The key ends up in the image	`.env` in `.dockerignore`; pass it with `--env-file` at run time; check with `docker history`
The eval score doesn't drop after the sabotage	A rule is too loose, or another agent repaired the damage — tighten the rule, or break something no later agent can fix

Built During These Documents

12_production_engineering → 13_testing_evaluation_observability (on top of Docs 01-11 and Project 4)

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks — and draw one request's path: middleware → rate limit → database → team → database → reply.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 answers over HTTP, Step 2 saves every run, Step 3 runs in a container behind a rate limit, Step 4 measures the articles themselves. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command runs from the **repo root** with the venv active.

How testing works in this project

Story: ContentForge Pro has two kinds of promise — the *service* behaves (right status codes, saved runs, a rate limit) and the *articles* stay good. They need two kinds of test, so every step adds the ones for the layer it builds. **If not:** a prompt edit that drops the `[source]` tags would pass every API test and ship.

- **Three layers of test:**
  - `tests/unit/` — Project 4's plain OK/WRONG scripts (the settings, the lookup tool, the routing rules). No key, no cost. **When:** after any change to the file they test.
  - `tests/api/test_routes.py` — pytest + `TestClient`, with the team **faked** by `monkeypatch`. Every route and status code, no model calls. **When:** after any change to `api/`.
  - `tests/integration/` and `evals/` — the **real** team and the real judge. Real API calls. **When:** at the end of a step, and before a prompt change.
- **Two test styles, on purpose:** Project 4's tests are plain scripts that print `OK` / `WRONG` — carried in unchanged, still run with `python -m tests.unit.test_config`. The **new** tests this project adds use pytest and `assert`, the style Doc12 and Doc13 teach, run with `pytest tests/api`. Never rewrite Project 4's tests into pytest.
- **Fake the team for API tests:** `monkeypatch.setattr(routes, "run_pipeline", fake)` swaps the one door into the team for a fixed answer — so the API's own behavior is tested with no model calls and the same result every run.
- **Calibrate the judge before trusting it:** `tests/integration/test_judge.py` gives the judge answers whose verdict you already know; a judge that approves everything is caught there.
- **Run from the repo root.** Plain scripts: `python -m tests.unit.test_config`. pytest: `pytest tests/api -v`.
- **Got a failure?** Read that one test, then the code it calls. Fix the code — change the test only if the test checks the wrong thing.

Step 1 — ContentForge, Reachable Over HTTP for the First Time

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 1 of 4

- **Where you are now:** Setup is done — Project 4 runs in the new repo, but only from the terminal.
- **What's new in this step:** `pipeline.py` (the one door into the team), the `api/` package (`schemas.py`, `routes.py`, `main.py`), and `tests/api/test_routes.py` with the team faked.
- **What stays the same:** every Project 4 file.

- **Why:** everything later builds on the HTTP layer. Prove it alone — no database, no Docker — so a later bug can't hide in it.
- **How:** `POST /run-task` → our own 400 for a blank topic → `run_pipeline(topic)` → a checked `TaskResponse`; any crash → `logger.exception()` + a plain 500. Tests: `TestClient` + `monkeypatch`.
- **Working after this step:** `uvicorn src.api.main:app --reload` answers `POST /run-task`; `pytest tests/api` passes with no model calls.

Read first: Doc12 — health route, Doc12 — chat route, Doc13 — testing layers, Doc13 — test a tool without the LLM (`monkeypatch`).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `pipeline.py`: `run_pipeline(topic)` runs the graph and returns `article`, `accepted`, `reason` and `steps`.
2. Write `api/schemas.py`: `TaskRequest(topic)` with `Field(min_length=1, max_length=200)`, and `TaskResponse(accepted, reason, article)`.
3. Write `api/routes.py`: an `APIRouter` with `GET /health` and `POST /run-task` (400 for a blank topic, a plain 500 for a crash).
4. Write `api/main.py`: `load_config()` first, then the app and `include_router(router)`.
5. Write `tests/api/test_routes.py`: one pytest test per case, with `monkeypatch` swapping `routes.run_pipeline` for a fake.
6. Run `pytest tests/api -v`, then start uvicorn and send one **real** topic from `/docs`.

The root files — `.gitignore`, `.env`, `.env.example`, `requirements.txt` and `LICENSE` — come from Setup; this step adds the files below.

Your files after Step 1:

```
<repo folder>/
├── src/
│   ├── api/
│   │   ├── __init__.py     (new)
│   │   ├── main.py         (new) load_config(), the app
│   │   ├── routes.py       (new) /health, /run-task
│   │   └── schemas.py      (new) TaskRequest, TaskResponse
│   └── pipeline.py         (new) run_pipeline(): the one door in
└── tests/
    └── api/
        └── test_routes.py  (new) 5 tests, team faked
```

Why these files (what and why):

- `api/main.py`
  - **What:** Creates `app = FastAPI(...)`, calls `load_config()` at startup, and includes the router.
  - **Why:** uvicorn needs one app object; a missing key stops the server at startup, not on the first request.
- `api/routes.py`
  - **What:** An `APIRouter` with `GET /health` and `POST /run-task`: a 400 for a blank topic, a plain 500 for a crash (the traceback stays in the server log).
  - **Why:** These are the addresses other programs call; each failure gets the right HTTP status, not a leaked traceback.
- `api/schemas.py`
  - **What:** `TaskRequest` (`topic` with a min and max length) and `TaskResponse` (`accepted`, `reason`, `article`).
  - **Why:** FastAPI checks every request, so bad input gets a 422 before it reaches the agents.
- `pipeline.py`
  - **What:** `steps_from_log(routing_log)` and `run_pipeline(topic)` — builds the graph, runs it, and returns `article`, `accepted`, `reason`, `steps`.
  - **Why:** The API and the evals both call this one function, so a test can swap it for a fake in one place.
- `tests/api/test_routes.py`
  - **What:** pytest tests with `TestClient` and `monkeypatch`: health, a normal run, a bad body (422), a blank topic (400), a crash (500) — the team faked.
  - **Why:** The API's behavior is tested quickly and for free, without model calls.

Tests for this step — write them yourself (the solution has the code, to compare):

- **test_health** — `tests/api/test_routes.py`
  - **Check:** `GET /health` returns 200 and `{"status": "ok"}`.
  - **Why:** the simplest proof the app starts and routes at all.
  - **When:** after any change to `api/main.py` or `api/routes.py`.
  - **If not:** a broken app object would fail every other test with no clear cause.
- **test_valid_request_is_answered** — `tests/api/test_routes.py`
  - **Check:** with the team faked, `POST /run-task` returns 200 and the fake article.
  - **Why:** proves the request shape, the route and the reply shape line up.
  - **When:** after changing the route or the schemas.
  - **If not:** a wrong field name would only be found by a real client.
- **test_bad_body_is_422** — `tests/api/test_routes.py`
  - **Check:** a body with no `topic` returns 422.
  - **Why:** bad input must be refused before it reaches the agents.
  - **When:** after changing `TaskRequest`.

    - **If not:** a malformed request would crash deep inside the team.
- **test_blank_topic_is_400** — `tests/api/test_routes.py`
    - **Check:** `{"topic": " "}` returns 400 with `topic cannot be empty`.
    - **Why:** `" "` passes Pydantic's length check but is still empty — our own guard.
    - **When:** after changing the blank-topic check.
    - **If not:** the team would run on an all-whitespace topic.
- **test_internal_error_is_a_plain_500** — `tests/api/test_routes.py`
    - **Check:** with a crashing fake, the route returns a plain 500, never the traceback.
    - **Why:** a client must never see file paths or `RuntimeError`.
    - **When:** after changing the `try/except` in the route.
    - **If not:** a crash would leak server internals to the caller.

Step 2 — Every Run Saved to a Database and Findable by ID

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 2 of 4

- **Where you are now:** after Step 1 the API answers, but a run's result exists only in the reply — once sent, it's gone.
- **What's new in this step:** the `db/` package (`database.py`, `models.py`), `GET /runs/{run_id}`, a `run_id` and `status` in every reply, and a request ID on every log line.
- **What stays the same:** the team, `pipeline.py`, the 400/500 rules, the request shape.
- **Why:** "what happened on that run yesterday?" needs a real answer — and a crashed run must leave an honest record, not vanish.
- **How:** `insert_started_run()` **before** the team runs → `finish_run()` or `fail_run()` after, each in its own `with conn:`; a middleware makes the request ID; `fetch_run()` + 404.
- **Working after this step:** every `POST /run-task` saves a row; `GET /runs/1` shows its input, article, routing log and status.

Read first: Doc12 — SQLite persistence, Doc12 — validation 422, Doc12 — chat route (Approach 4, the request ID).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `db/database.py`: `DB_PATH`, `get_connection()`, `insert_started_run()`, `finish_run()`, `fail_run()`, `fetch_run()`.
2. Write `db/models.py`: `Run`, the shape `GET /runs/{run_id}` returns.
3. Add `run_id` and `status` to `TaskResponse`.
4. Change `routes.py`: save `"started"` first, `"completed"` or `"failed"` after; the routing steps go in the `log` column; add `GET /runs/{run_id}` with a 404.

5. Add the request-ID middleware to `api/main.py`, and put `[request_id]` at the start of every log line.
6. Add tests: the row is saved, a 422 saves nothing, a crash is saved as `"failed"`, a wrong ID is a 404, every reply has `X-Request-ID`.

Your files after Step 2:

```
<repo folder>/
├── src/
│   ├── api/
│   │   ├── main.py         (changed) + request-ID middleware
│   │   ├── routes.py       (changed) saves runs, + /runs/{run_id}
│   │   └── schemas.py      (changed) + run_id, status
│   └── db/
│       ├── __init__.py     (new)
│       ├── database.py     (new) insert/finish/fail/fetch a run
│       └── models.py       (new) Run
└── tests/
    └── api/
        └── test_routes.py  (changed) + saved-row checks, 404
```

Why these files (what and why):

- `api/main.py`
  - **What:** Adds the `add_request_id` middleware — a short ID per request, on `request.state` and in the `X-Request-ID` header.
  - **Why:** every log line of one request can be found by its ID.
- `api/routes.py`
  - **What:** Now saves a `"started"` run before the team, `"completed"` / `"failed"` after, and adds `GET /runs/{run_id}` with a 404.
  - **Why:** every run leaves an honest record, even a crashed one.
- `api/schemas.py`
  - **What:** `TaskResponse` gains `run_id` and `status`.
  - **Why:** the client gets the saved run's ID to look it up later.
- `db/database.py`
  - **What:** `DB_PATH`, the table SQL, and `get_connection`, `insert_started_run`, `finish_run`, `fail_run`, `fetch_run`.
  - **Why:** every run is saved with its status, findable after a restart.
- `db/models.py`
  - **What:** `Run`, one saved run's shape (`id`, `input`, `output`, `log`, `status`, `created_at`).
  - **Why:** the database and the `GET /runs/{id}` reply agree on the same fields.

- `tests/api/test_routes.py`
  - **What:** the same tests, now also checking the saved rows, the 404, and the `X-Request-ID` header.
  - **Why:** persistence is part of the API's behavior, so it's tested the same way.

Tests for this step — write them yourself:

- **test_valid_request_is_answered (saved row)** — `tests/api/test_routes.py`
  - **Check:** after a run, `GET /runs/{run_id}` shows the input, the log and `status == "completed"`.
  - **Why:** proves the run was actually saved, not just returned.
  - **When:** after changing the save logic or `db/database.py`.
  - **If not:** a run that was never saved would still pass Step 1's test.
- **test_bad_body_is_422 (nothing saved)** — `tests/api/test_routes.py`
  - **Check:** a 422 leaves the row count unchanged.
  - **Why:** a rejected request must not create a database row.
  - **When:** after changing where the save happens in the route.
  - **If not:** half-formed "started" rows would pile up from bad requests.
- **test_internal_error_is_a_plain_500 (saved as failed)** — `tests/api/test_routes.py`
  - **Check:** a crash leaves one more `"failed"` row.
  - **Why:** a crashed run must leave an honest record, not vanish.
  - **When:** after changing the `except` branch.
  - **If not:** failures would be invisible after the fact.
- **test_unknown_run_is_404** — `tests/api/test_routes.py`
  - **Check:** `GET /runs/999999` returns 404.
  - **Why:** an unknown ID is a clean "not found", not a 500.
  - **When:** after changing `get_run` or `fetch_run`.
  - **If not:** a missing run could crash the route.
- **test_every_reply_has_a_request_id** — `tests/api/test_routes.py`
  - **Check:** every reply carries an 8-character `X-Request-ID`.
  - **Why:** a caller can quote it when reporting a problem.
  - **When:** after changing the middleware.
  - **If not:** interleaved request logs would be impossible to separate.

Step 3 — Containerized and Rate-Limited: Runs the Same Everywhere, Survives Abuse

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 3 of 4

- **Where you are now:** after Step 2 the API saves every run — but it runs only on your laptop, and anyone can call it as often as they like.
- **What's new in this step:** a per-caller rolling-window rate limit (`429`), a `Dockerfile` , a `.dockerignore` , and a test for the 429.
- **What stays the same:** the routes' other rules, the database, the team.
- **Why:** each run costs five agents' worth of model calls — one script in a loop could empty your budget. And "it works on my machine" isn't something you can hand to anyone.
- **How:** `check_rate_limit(http_request.client.host)` first in `POST /run-task` ; the image installs the requirements, copies the code, and starts uvicorn on `0.0.0.0` ; the key arrives only with `docker run --env-file .env` .
- **Working after this step:** the API runs from a container; the sixth quick request from one caller gets a 429; `docker run` without a key stops at once with a clear error.

Read first: Doc12 — dead connection and container, Doc12 Build Task (the rate limiter).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add the rate limiter to `routes.py` : `RATE_LIMIT` , `RATE_WINDOW_SECONDS` , `request_times` , `check_rate_limit(caller_key)` — and write its limitation in a comment.
2. Call it first in `POST /run-task` , with `http_request.client.host` .
3. In the tests, reset `request_times` in `use_fake_team()` , and add a 429 test.
4. Write the `Dockerfile` and `.dockerignore` .
5. `docker build` , `docker run` with `--env-file .env` , and `curl localhost:8000/health` . Check `docker history` has no key, and that `docker run` **without** the key fails loudly.

Your files after Step 3:

```
<repo folder>/
├── src/
│   └── api/
│       └── routes.py       (changed) + check_rate_limit(), 429
├── tests/
│   └── api/
│       └── test_routes.py  (changed) + the 429 test
├── .dockerignore           (new) no .env, no runs.db in the image
└── Dockerfile              (new) the API's image
```

Why these files (what and why):

- `api/routes.py`
  - **What:** Adds `RATE_LIMIT` , `RATE_WINDOW_SECONDS` , `request_times` and `check_rate_limit(caller_key)` , called first in `POST /run-task` .

    - **Why:** a caller over the limit gets a 429 before any paid model call.
- `tests/api/test_routes.py`
    - **What:** resets the limiter in `use_fake_team()` and adds a 429 test.
    - **Why:** the rate limit is part of the route's behavior, tested for free.
- `.dockerignore`
    - **What:** leaves `.env`, `.git`, the venv, caches, `runs.db` and `report.json` out of the image.
    - **Why:** your key and local data never end up inside the image.
- `Dockerfile`
    - **What:** `FROM python:3.11-slim`, install the requirements, copy the code, `EXPOSE 8000`, and start uvicorn on `0.0.0.0`.
    - **Why:** the API runs the same on any machine, with the key passed in only at run time.

Tests for this step — write them yourself:

- **test_over_the_rate_limit_is_429** — `tests/api/test_routes.py`
    - **Check:** with `RATE_LIMIT` set to 2, three quick calls return `[200, 200, 429]`.
    - **Why:** the limiter must actually block the call over the limit.
    - **When:** after changing `check_rate_limit` or the window.
    - **If not:** a loop of requests could run up the model bill unchecked.
- **reset `request_times` per test** — `tests/api/test_routes.py`
    - **Check:** `use_fake_team()` clears `routes.request_times`, so earlier tests don't count against a later one.
    - **Why:** shared limiter memory would make tests fail at random.
    - **When:** whenever you add a test that sends several requests.
    - **If not:** test order would change the results.

Step 4 — A Test Suite That Catches a Bad Change Before It Ships (final)

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 4 of 4

- **Where you are now:** after Step 3 the service is solid — but its tests fake the team, so they can't see a **worse article**.
- **What's new in this step:** the `evals/` package (`tasks.py`, `judge.py`, `run_eval.py`, `regression_check.py`) and `tests/integration/test_judge.py`. Project 4's tests stay exactly as they are.
- **What stays the same:** the API, the database, Docker, the team.
- **Why:** a prompt edit that quietly drops the `[source]` tags would pass every API test. Only a suite that scores the real output — and has been seen to fail — catches it.

- **How:** 10 fixed topics with checks written first → `run_pipeline()` → a free code check, or the judge for meaning → a score and `report.json`; then break Research's prompt on purpose and prove the score falls below the bar.
- **Working after this step:** the full project — the unit and API tests pass for free, `run_eval` scores 10/10, and `regression_check` reports the drop.

Read first: Doc13 — a normal, exact test, Doc13 — fix a flaky test, Doc13 — score an answer, Doc13 — regression catch, Doc13 Build Task.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Leave Project 4's tests (`tests/unit/*` , `tests/integration/test_live_agents.py`) exactly as they are — this step only adds tests.
2. Write `evals/tasks.py` — 10 topics: exact `[source]` tags, judge rules, and unknown topics that must give no article. **Write them before running anything.**
3. Write `evals/judge.py` (Doc13's judge) and `evals/run_eval.py` (calls `run_pipeline()` , prints the score, saves `report.json`).
4. Write `tests/integration/test_judge.py` : the judge must PASS a good article and FAIL two bad ones.
5. Write `evals/regression_check.py` , run it, and delete the words it asks for from `research.txt` when it pauses. Put them back afterwards.

Your files after Step 4 (final): the full tree is in Project Structure. What changed:

```
<repo folder>/
├── src/
│   └── evals/
│       ├── __init__.py          (new)
│       ├── judge.py             (new) score_with_judge()
│       ├── regression_check.py  (new) sabotage, prove the drop
│       ├── run_eval.py          (new) the score, report.json
│       └── tasks.py             (new) 10 topics, checks first
└── tests/
    └── integration/
        └── test_judge.py        (new) calibration set
```

Why these files (what and why):

- `evals/judge.py`
  - **What:** `JudgeResult` , `call_model()` , `build_judge_prompt()` , and `score_with_judge()` — PASS/FAIL from the first line.
  - **Why:** qualities like clarity can't be checked by exact code; a judge with written rules can.

- `evals/regression_check.py`
  - **What:** `PASS_BAR`, a `SABOTAGE` text, `score()`, `find_newly_failed()`, and `run_regression_check()`.
  - **Why:** it proves the eval suite really catches a bad change.
- `evals/run_eval.py`
  - **What:** `run_one_task()` and `run_eval_suite()` — run all tasks, print the score, save `report.json`.
  - **Why:** one score for the whole system, to compare two versions.
- `evals/tasks.py`
  - **What:** rule constants and a `TASKS` list of 10 dicts (exact text, judge rules, or no-article).
  - **Why:** the same fixed test set every time, for a fair comparison.
- `tests/integration/test_judge.py`
  - **What:** a pytest calibration test — the judge must PASS a good article and FAIL two bad ones.
  - **Why:** you can only trust the judge after checking it on known answers.

Tests for this step — write them yourself:

- **test_judge_calibration** — `tests/integration/test_judge.py`
  - **Check:** the judge PASSes a titled, tagged article and FAILs an off-topic one and one with no tags.
  - **Why:** a judge that approves everything would make every eval run look perfect.
  - **When:** whenever `judge.py`'s prompt changes.
  - **If not:** every later score would be meaningless.
- **the regression check** — `src/evals/regression_check.py`
  - **Check:** the score is ≥ `PASS_BAR` before the sabotage and below it after.
  - **Why:** an eval that can never fail proves nothing.
  - **When:** after changing the tasks or the rules.
  - **If not:** a loose rule would let a real regression ship unnoticed.

Final Deliverable: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Project 4's five agents behind a FastAPI service: typed requests, safe errors, every run saved and findable by ID, request-ID logs, a per-caller rate limit, a Docker image with no secrets inside, API tests with the team faked, and an eval suite proven to catch a bad prompt change.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 4 is done, this is how the finished project is run, tested and configured.

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key; the app will not start without it
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

Setting	Where	Default
Chat model	`src/models/llm.py` → `MODEL_NAME`	`gpt-4o-mini`
Pass mark	`src/agents/reviewer_agent.py` → `PASS_SCORE`	`7` (out of 10)
Revision limit	`src/agents/supervisor.py` → `MAX_REVISIONS`	`3`
Topic length	`src/api/schemas.py` → `TaskRequest`	1-200 characters
Rate limit	`src/api/routes.py` → `RATE_LIMIT`, `RATE_WINDOW_SECONDS`	5 per 60 s, per caller
Database file	`src/db/database.py` → `DB_PATH`	`runs.db`
Eval pass bar	`src/evals/regression_check.py` → `PASS_BAR`	`0.8`
Known facts	`data/sample/facts.json`	electric bikes, remote work, solar panels

Usage

Run from the repo root:

```
uvicorn src.api.main:app --reload     # the API on port 8000
python main.py                        # the terminal app
```

Open `http://localhost:8000/docs` to try every route in the browser.

Route	What it does
`GET /health`	`{"status":"ok"}`
`POST /run-task`	Body `{"topic": "..."}`; returns `run_id`, `status`, `accepted`, `reason`, `article`
`GET /runs/{run_id}`	The saved run: `input`, `output`, `log`, `status`, `created_at`; 404 if unknown

In a container:

```
docker build -t contentforge-pro .
docker run -p 8000:8000 --env-file .env contentforge-pro
curl localhost:8000/health
```

Running the Tests

Run from the repo root:

```
# free — no model calls
python -m tests.unit.test_config
python -m tests.unit.test_lookup
python -m tests.unit.test_supervisor
pytest tests/api -v

# real API calls — needs a valid OPENAI_API_KEY
python -m tests.integration.test_live_agents
pytest tests/integration/test_judge.py -v
```

The unit tests (Project 4's plain OK/WRONG scripts) check the settings, the lookup tool and every routing rule. The API tests check every route and status code — 200, 400, 404, 422, 429 and a plain 500 — and the saved rows, with the team replaced by a fake. The integration tests run the real team and check the judge on answers with a known verdict.

Article quality (real API calls):

```
python -m src.evals.run_eval           # score + report.json
python -m src.evals.regression_check   # prove a bad change drops it
```

`run_eval` runs 10 fixed topics — exact `[source]` checks, judge rules and unknown topics — and prints the score. `regression_check` runs it, pauses while you remove the `[source]` rule from Research's prompt, runs it again, and reports whether the score fell below the bar.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] Project 4's files are unchanged; the API calls the team only through `run_pipeline()`
- [ ] Code lives in `src/`, `main.py` at the repo root; the API starts with `uvicorn src.api.main:app`
- [ ] `POST /run-task` has typed shapes: 422 for a bad body, 400 for a blank topic, a plain 500 for a crash (the traceback only in the server log)

- [ ] Every run is saved (`"started"` → `"completed"` / `"failed"`) and found with `GET /runs/{run_id}`; a wrong ID is a 404
- [ ] Every log line for a request starts with the same request ID, also sent back as `X-Request-ID`
- [ ] A per-caller rate limit returns 429, and its limitation is written in the code
- [ ] `docker run` works with the key passed at run time; `docker history` shows no key; no key → a clear startup error
- [ ] The unit scripts and `pytest tests/api` pass with no model calls
- [ ] The eval suite scores the real articles, and the regression check proves a sabotaged prompt drops the score
- [ ] The judge passes its own calibration test
- [ ] `.env`, `runs.db` and `report.json` are not in git
- [ ] You can defend the trade-offs: SQLite vs. Postgres, the in-memory rate limit, the synchronous route

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and put your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status                  # .env, .venv/, runs.db must NOT appear
git add .
git commit -m "ContentForge Pro: API, run history, Docker, test suite"
```

3. On GitHub, create a new repo named exactly `ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of README.md. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started. Track your progress in ../PROGRESS.md.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**